MASTER HANDBOOK

📘 DSA Revision Notes

Placement 2026 · Week 1 · Arrays, Searching & Prefix Sum

- ✅ Beginner-friendly explanations with interview-grade depth
- ✅ Brute Force → Optimal walkthroughs for every problem
- ✅ Full dry runs, pattern recognition cues & common mistakes
- ✅ Complexity Cheat Sheet, Python Shortcuts & Final Revision Sheet

Continuously updated companion for LeetCode practice and interview preparation

Table of Contents

Big-O Notation

FOUNDATIONS GROWTH RATE ANALYSIS

📖 Definition

Big-O notation describes how the running time (or memory usage) of an algorithm grows as the input size `N` grows. It does not measure exact seconds — it measures the **shape of growth**. This lets us compare algorithms independent of the machine they run on.

💡 Problem Statement

Given any piece of code, we want a quick way to answer: "If I double the input size, how much slower does this get?" Big-O gives us a label (O(1), O(N), O(N²)...) that answers exactly this.

🧠 Intuition

Think of Big-O as a worst-case "speed category", not a stopwatch reading. Two algorithms with the same Big-O can have different real run times, but as N becomes very large, the one with the better (lower) Big-O always wins. Interviewers care about Big-O because it predicts how your code behaves on large, real-world inputs — not just the small example in front of you.

Common Complexities (Best → Worst)

```
O(1)  <  O(log N)  <  O(N)  <  O(N log N)  <  O(N²)  <  O(N³)  <  O(2^N)  <  O(N!)
```

🚀 Core Rules

Rule 1 — Ignore Constants

```
O(5N)     = O(N)
O(100N)   = O(N)
O(N/2)    = O(N)
```

Constants don't change the *shape* of growth, so we drop them.

Rule 2 — Keep the Highest Growing Term

```
O(N² + N + 100)      = O(N²)
O(N² + N log N)      = O(N²)
O(N + log N)         = O(N)
```

As N grows huge, smaller terms become insignificant compared to the dominant term.

Rule 3 — Loops: Add vs Multiply

Separate (sequential) loops → ADD

```
for i in range(n):
    ...
for j in range(n):
    ...
# O(N) + O(N) = O(N)
```

Nested loops → MULTIPLY

```
for i in range(n):
    for j in range(n):
        ...
# O(N) x O(N) = O(N-squared)
```

🔍 Complexity Patterns You Must Recognize

Code Pattern	Complexity	Why
`arr[0]`, `arr[n-1]`	O(1)	Direct index access, no traversal
Single `for` loop over N elements	O(N)	One pass over the input
`i *= 2` or `i //= 2` until base case	O(log N)	Search space halves every step
Outer loop N, inner loop doubles/halves (logN)	O(N log N)	N outer steps × log N inner steps
Two nested loops over N	O(N²)	N × N total operations
`for j in range(i)` (triangular loop)	O(N²)	0+1+2+...+(N-1) = N(N-1)/2
Inner loop runs a fixed number of times (e.g. 100)	O(N)	A constant bound doesn't depend on N
Two nested "while" loops each doubling (logN × logN)	O(log² N)	Outer log N, inner log N

> 🚀 **Triangle Pattern (very common in interviews)**
>
> `for i in range(n): for j in range(i):` performs 0+1+2+...+(N-1) = N(N-1)/2 operations → still **O(N²)**, even though it "looks" smaller than a full nested loop.

⏱ Best / Average / Worst Case

Linear Search on an unsorted array: **Best** = O(1) (target is the first element), **Average** = O(N), **Worst** = O(N). Interviews almost always discuss the **Worst Case**, because that's the guarantee your code must hold for any input.

💻 Space Complexity

```
a = 10
b = 20
# Fixed variables -> O(1)
```

```
arr = []
for i in range(n):
    arr.append(i)
# Growing array -> O(N)
```

⚠ Common Mistakes

- Forgetting to drop constants (writing O(2N) instead of simplifying to O(N))
- Confusing a "small-looking" nested loop (triangle pattern) with O(N) — it is still $O(N^2)$
- Assuming average case = worst case for unsorted-array searches
- Ignoring space used by recursion call stacks

🎯 Interview Shortcuts (Memorize These)

Pattern	Complexity
`arr[index]`	O(1)
Single loop	O(N)
Nested loop (2 levels)	$O(N^2)$
Nested loop (3 levels)	$O(N^3)$
`i *= 2` or `i //= 2`	O(log N)
N + N (sequential loops)	O(N)
N × log N (sort + scan)	O(N log N)
0+1+2+...+N	$O(N^2)$

🎤 Interview Questions

- **Why do we drop constants in Big-O?** Because Big-O measures growth trend, not exact speed — constants don't change how steeply the curve rises as $N \to \infty$.
- **Why is the triangular nested loop still $O(N^2)$?** Because the sum of the first N integers is proportional to N^2, even though the inner loop shrinks each time.
- **When is Worst Case more important than Average Case?** When correctness/performance must be guaranteed for every possible input, which is the standard interview assumption.

🔄 Where You'll Use This

Every single problem in this handbook is analyzed using these exact rules. Big-O is the language used throughout — make sure this page is solid before moving on.

"Big-O tells you how an algorithm scales — not how fast it runs today, but how painfully slow it could get on a huge input tomorrow."

Reverse Array

EASY TWO POINTERS ARRAYS

📖 Definition

Reversing an array means rearranging its elements so the last element becomes the first, the second-last becomes the second, and so on — done in-place, without using extra memory.

💡 Problem Statement

Given an array, reverse the order of its elements.

```
Input:  [1,2,3,4,5]
Output: [5,4,3,2,1]
```

🧠 Intuition

You don't need a new array to reverse — you only need to swap elements that are mirror images of each other across the center. The first and last swap, the second and second-last swap, and so on, until the two pointers meet in the middle.

🐢 Brute Force Approach

Idea: Traverse from the last index to the first, and append each element into a new result array.

```
arr = [1,2,3,4,5]
res = []

for i in range(len(arr)-1, -1, -1):
    res.append(arr[i])
```

Time Complexity: O(N) | **Space Complexity:** O(N)

Why it's inefficient: It does the job correctly but wastes O(N) extra memory for a problem that can be solved with zero extra space.

🚀 Optimal Approach — Two Pointers

Main idea: Place one pointer at the start (`left`) and one at the end (`right`). Swap the elements they point to, then move `left` forward and `right` backward. Stop when they meet.

Pattern used: Two Pointers (converging from both ends) — this pattern appears constantly in array problems.

Why it is better: No extra array is needed; every swap permanently places two elements in their final position.

🔍 Dry Run

```
arr = [1, 2, 3, 4, 5]
left = 0, right = 4

Swap arr[0] and arr[4]  ->  [5, 2, 3, 4, 1]
left = 1, right = 3

Swap arr[1] and arr[3]  ->  [5, 4, 3, 2, 1]
left = 2, right = 2  -> left == right, STOP

Final: [5, 4, 3, 2, 1]
```

💻 Code

```python
left = 0
right = len(arr) - 1

while left < right:
    arr[left], arr[right] = arr[right], arr[left]
    left += 1
    right -= 1
```

🔎 Code Explanation

- `left` and `right` mark the two ends that need to be swapped next.
- The loop condition `left < right` guarantees we stop exactly when the pointers meet (or cross), so every element is swapped exactly once.
- The tuple-swap line exchanges values without needing a temporary variable.

✅ Why This Works

Reversing is just mirroring positions: index `i` should end up holding the value that was originally at index `N-1-i`. Swapping from both ends toward the center achieves exactly this mirroring in a single pass.

⏱ Complexity Analysis

Approach	Time	Space
Extra Array	O(N)	O(N)
Two Pointers (Optimal)	O(N)	O(1)

Each pointer moves at most N/2 steps, so the total work is linear, but no extra memory is allocated.

🎯 Pattern Recognition

🧩 Trigger Words

"Reverse", "in-place", "swap from both ends" → **Two Pointers**. This exact pointer-converging idea reappears in Rotate Array, Palindrome Check, and Valid Palindrome problems.

⚠ Common Mistakes

- Using `left <= right` instead of `left < right` (causes an unnecessary extra "self-swap" in odd-length arrays)
- Forgetting to move both pointers after the swap (infinite loop)
- Creating a brand-new array when the problem explicitly asks for in-place modification

🎤 Interview Questions

- **Why is the in-place approach O(1) space?** Because we never allocate a new data structure — we only swap existing elements.
- **What happens if the array has an odd number of elements?** The middle element never gets swapped because `left` and `right` meet exactly on it — it's already in its correct position.

🔄 Similar Problems

- 344. Reverse String
- 189. Rotate Array
- 125. Valid Palindrome
- 541. Reverse String II

"Reverse = swap mirror positions from both ends until the pointers meet."

Rotate Array

MEDIUM | ARRAY REVERSAL | LEETCODE 189

📖 Definition

Rotating an array to the right by `k` steps means every element moves `k` positions to the right, and elements that fall off the end wrap around to the front.

💡 Problem Statement

```
Input:  nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
```

🧠 Intuition

The key building block is the modulo operator `%`, which wraps an index back to the start whenever it overflows. For any index `i`, its new position after rotating right by `k` is `(i + k) % n`. The optimal solution avoids building a fresh array by cleverly using three reversals instead.

> 💡 **Modulo Refresher**
>
> `7 % 7 = 0`, `8 % 7 = 1`, `9 % 7 = 2`, `10 % 7 = 3` — modulo is how we "wrap around" an array boundary.

🐢 Brute Force Approach — Extra Array

Idea: Compute each element's new wrapped index directly using the formula above and place it into a new array.

```
res = [0] * len(nums)

for i in range(len(nums)):
    res[(i + k) % len(nums)] = nums[i]

nums[:] = res
```

Time Complexity: O(N) | **Space Complexity:** O(N)

Why it is inefficient: Correct and fast, but uses an extra array when the problem can be solved in-place.

🚀 Optimal Approach — Reverse Method

Main idea: Three reversals achieve a full rotation: (1) reverse the entire array, (2) reverse the first `k` elements, (3) reverse the remaining elements.

Pattern used: Array Reversal (reusing the Two-Pointer reverse technique three times).

Why it is better: Zero extra memory — everything happens through in-place swaps.

🔍 Dry Run

```
nums = [1,2,3,4,5,6,7], k = 3

Step 1: Reverse entire array
1 2 3 4 5 6 7  ->  7 6 5 4 3 2 1

Step 2: Reverse first k=3 elements
[7 6 5] 4 3 2 1  ->  [5 6 7] 4 3 2 1
Array now: 5 6 7 4 3 2 1

Step 3: Reverse remaining n-k=4 elements
5 6 7 [4 3 2 1]  ->  5 6 7 [1 2 3 4]

Final: [5, 6, 7, 1, 2, 3, 4]
```

💻 Code

```python
def reverse(arr, left, right):
    while left < right:
        arr[left], arr[right] = arr[right], arr[left]
        left += 1
        right -= 1

def rotate(nums, k):
    n = len(nums)
    k = k % n          # handle k > n

    reverse(nums, 0, n - 1)      # Step 1: reverse all
    reverse(nums, 0, k - 1)      # Step 2: reverse first k
    reverse(nums, k, n - 1)      # Step 3: reverse the rest
```

🔎 Code Explanation

- `k = k % n` handles the edge case where `k` is larger than the array length (rotating by `n` is the same as rotating by 0).
- Reversing the whole array puts every element in reverse order, but the desired "blocks" (last k elements, first n-k elements) are now adjacent at the front and back.
- Reversing each block individually un-reverses it locally, leaving the blocks themselves in the correct rotated order.

✅ Why This Works

Rotating right by `k` is equivalent to moving the last `k` elements to the front. A full reversal places those last `k` elements at the front — but backwards. Reversing that front block fixes their internal order; reversing the remaining block fixes the rest. Three reversals = one rotation.

⏱ Complexity Analysis

Approach	Time	Space
Extra Array	O(N)	O(N)
Reverse Method (Optimal)	O(N)	O(1)

Each reversal is O(length of that segment); three reversals together still touch each element a constant number of times → O(N) overall.

🎯 Pattern Recognition

🧩 Trigger Words

"Rotate array", "shift elements", "wrap around" → **Reverse Technique** or modulo-index mapping.

⚠ Common Mistakes

- Forgetting `k = k % n` when `k > n` (causes index errors or wasted rotations)
- Reversing blocks in the wrong order (must reverse the WHOLE array first)
- Off-by-one errors when slicing the "first k" vs "remaining n-k" segments

🎤 Interview Questions

- **Why does reversing three times work?** Because reversing the whole array correctly groups the wrapped elements together; reversing each sub-block then restores local order.
- **What if k is negative or larger than n?** Apply `k = k % n` first to normalize it into the valid range `[0, n-1]`.

🔄 Similar Problems

- 61. Rotate List
- 48. Rotate Image
- 396. Rotate Function
- 1. Reverse Array (prerequisite)

"Rotate = Reverse all, then Reverse the two halves back into shape."

Move Zeroes

EASY TWO POINTERS LEETCODE 283

📖 Definition

Move Zeroes asks us to push every zero in an array to the end while keeping the relative order of all non-zero elements unchanged — done in-place.

💡 Problem Statement

```
Input:  [0,1,0,3,12]
Output: [1,3,12,0,0]
```

🧠 Intuition

Imagine walking through the array and "collecting" every non-zero number into the front of the array, in the order you found them. A single pointer (`right`) tracks the next free slot at the front for the next non-zero value. Once all non-zero values are placed, every remaining position must be zero.

🐢 Brute Force Approach

Idea: Build a new array containing only the non-zero elements (in order), then append zeros at the end.

Time Complexity: O(N) | **Space Complexity:** O(N)

Why it is inefficient: Correct, but allocates an entire second array when the rearrangement can be done in-place.

🚀 Optimal Approach — Two Pointers (Overwrite)

Main idea: Use a pointer `right` that always points to "the next free position for a non-zero element." Scan left to right; whenever a non-zero element is found, place it at `right` and advance `right`. After the scan, fill everything from `right` to the end with zero.

Pattern used: Two Pointers (fast scanner + slow writer).

Why it is better: O(1) extra space — the array is rearranged using itself as storage.

🔍 Dry Run

```
nums = [0, 1, 0, 3, 12]
right = 0

i=0: nums[0]=0  -> skip
i=1: nums[1]=1  -> non-zero -> nums[right]=nums[0]=1, right=1
i=2: nums[2]=0  -> skip
i=3: nums[3]=3  -> non-zero -> nums[right]=nums[1]=3, right=2
i=4: nums[4]=12 -> non-zero -> nums[right]=nums[2]=12, right=3

Array after copy step: [1, 3, 12, 3, 12]

Fill remaining positions (index 3 to end) with 0:
[1, 3, 12, 0, 0]
```

💻 Code

```python
right = 0

for i in range(len(nums)):
    if nums[i] != 0:
        nums[right] = nums[i]
        right += 1

for i in range(right, len(nums)):
    nums[i] = 0
```

🔎 Code Explanation

- The first loop compacts every non-zero value toward the front, preserving their relative order — `right` always equals "how many non-zero elements placed so far".
- After the first loop, everything from index `right` onward is "leftover" — those positions must become zero.
- The second loop fills exactly those leftover positions with zero.

✅ Why This Works

Because we only ever write a value at index ≤ the index we're currently reading from (`right ≤ i` always holds), we never overwrite data we still need to read. This makes the in-place compaction safe.

⏱️ Complexity Analysis

Approach	Time	Space
Brute Force (new array)	O(N)	O(N)
Two Pointers (Optimal)	O(N)	O(1)

🎯 Pattern Recognition

🧩 Trigger Words

"Move to the end", "maintain relative order", "in-place rearrangement" → **Two Pointers (fast scanner / slow writer)**. The exact same template solves "Remove Element" and "Remove Duplicates from Sorted Array".

⚠ Common Mistakes

- Forgetting the second loop, leaving stale duplicate values instead of zeros at the tail
- Incrementing `right` even when the current element is zero (breaks the compaction)
- Swapping instead of overwriting when overwriting is simpler and sufficient here

🎤 Interview Questions

- **Why does `right` never overtake `i`?** Because `right` only increases when a non-zero value is found, and that can happen at most as often as `i` advances.
- **Could you solve this with swapping instead of overwriting?** Yes — swap `nums[i]` and `nums[right]` whenever `nums[i] != 0`; this also avoids needing the cleanup loop.

🔄 Similar Problems

- 27. Remove Element
- 26. Remove Duplicates from Sorted Array
- 75. Sort Colors (Dutch National Flag)

"Compact non-zeros to the front using a write-pointer, then flood the rest with zeros."

Two Sum

EASY HASHMAP LEETCODE 1

📖 Definition

Two Sum asks us to find the indices of two numbers in an array that add up exactly to a given target value.

💡 Problem Statement

```
Input:  nums = [2,7,11,15], target = 9
Output: [0,1]   (because nums[0] + nums[1] = 2 + 7 = 9)
```

🧠 Intuition

Instead of checking every pair of numbers, ask a sharper question for each number: *"What number would I need to complete the target, and have I already seen it?"* A hashmap lets us answer "have I seen X?" in O(1), turning a quadratic search into a linear one.

🐢 Brute Force Approach

Idea: Check every possible pair `(i, j)` and test if they sum to the target.

```
for i in range(len(nums)):
    for j in range(i+1, len(nums)):
        if nums[i] + nums[j] == target:
            return [i, j]
```

Time Complexity: O(N²) | **Space Complexity:** O(1)

Why it is inefficient: For every element, we re-scan the rest of the array — wasted, repeated comparisons.

🚀 Optimal Approach — HashMap / Complement Technique

Main idea: While traversing, compute `complement = target - current_number`. If the complement already exists in our hashmap, we've found our pair immediately. Otherwise, store the current number and its index for future lookups.

Pattern used: HashMap (store-and-check / complement technique).

Why it is better: Each element is processed once, and hashmap lookups are O(1) on average, giving overall O(N).

🔍 Dry Run

```
nums = [2, 7, 11, 15], target = 9
seen = {}

i=0: num=2,  complement = 9-2 = 7   -> not in seen -> seen = {2: 0}
i=1: num=7,  complement = 9-7 = 2   -> 2 IS in seen (index 0)
             -> return [0, 1]
```

💻 Code

```python
def twoSum(nums, target):
    seen = {}                       # value -> index

    for i, num in enumerate(nums):
        complement = target - num

        if complement in seen:
            return [seen[complement], i]

        seen[num] = i

    return []
```

🔎 Code Explanation

- `seen` maps every number we've already visited to its index.
- `complement` is the exact value that, combined with the current number, would equal the target.
- Checking `complement in seen` is an O(1) hashmap lookup — this is what replaces the inner loop from brute force.
- We only store the current number *after* checking, so we never match a number with itself.

✅ Why This Works

If a valid pair `(a, b)` exists such that `a + b = target`, then when we reach `b` in the traversal, `a` must already be sitting in the hashmap (since `a` appears earlier as required by the pair). So the lookup is guaranteed to succeed at the correct moment.

⏱ Complexity Analysis

Approach	Time	Space
Brute Force	O(N²)	O(1)
HashMap (Optimal)	O(N)	O(N)

We trade O(N) extra space for dropping one full nested loop — a very common and worthwhile trade-off in interviews.

🎯 Pattern Recognition

🧩 Trigger Words

"Pair sum", "have I seen this value", "complement", "two numbers that add up to" → **HashMap**.

⚠ Common Mistakes

- Storing the current number in the map *before* checking for its complement (can wrongly pair an element with itself)
- Returning values instead of indices (read the problem statement carefully)
- Using nested loops out of habit when a single pass with a hashmap is sufficient

🎤 Interview Questions

- **Why HashMap instead of nested loops?** Because hashmap lookups are O(1) on average, collapsing the search for a complement from O(N) to O(1) per element.
- **What if there are duplicate values in nums?** The "check before insert" order in our code still works, since we only ever look back at values seen strictly before the current index.
- **What if no valid pair exists?** Return an empty list (or as specified by the problem) — always handle this edge case explicitly.

🔄 Similar Problems

- 15. 3Sum
- 167. Two Sum II – Input Array Is Sorted
- 1. Contains Duplicate
- 49. Group Anagrams

"Two Sum = for every number, ask 'have I already seen its complement?' using a HashMap."

Contains Duplicate

EASY HASHSET LEETCODE 217

📖 Definition

This problem asks whether any value appears at least twice in an array of integers.

💡 Problem Statement

```
Input:  nums = [1,2,3,1]
Output: True   (because 1 appears twice)
```

🧠 Intuition

The question is really: *"Have I seen this value before, anywhere earlier in the array?"* A set is the perfect structure for tracking "have I seen this" because membership checks are O(1) on average.

🐢 Brute Force Approach

Idea (Nested Loops): Compare every element with every other element.

```
for i in range(len(nums)):
    for j in range(i+1, len(nums)):
        if nums[i] == nums[j]:
            return True
return False
```

Time Complexity: O(N²) | **Space Complexity:** O(1)

> **⚠ A Sneaky "Optimization" That Isn't**
>
> A common first attempt is `nums.count(num) >= 2` inside a loop. This *looks* shorter, but `count()` itself scans the whole list, so the total work is still N × N = **O(N²)** — it just hides the nested loop instead of removing it.

🚀 Optimal Approach — HashSet

Main idea: Traverse the array once. For each element, check if it's already in a set. If yes, we found a duplicate. If no, add it to the set and continue.

Pattern used: HashSet ("seen before" tracking).

Why it is better: Each membership check and insertion is O(1) on average, so the whole scan is O(N).

🔍 Dry Run

```
nums = [1, 2, 3, 1]
seen = {}

1 -> not in seen -> seen = {1}
2 -> not in seen -> seen = {1, 2}
3 -> not in seen -> seen = {1, 2, 3}
1 -> ALREADY in seen -> return True
```

💻 Code

```python
def containsDuplicate(nums):
    seen = set()

    for num in nums:
        if num in seen:
            return True
        seen.add(num)

    return False
```

🔎 Code Explanation

- `seen` stores every distinct value encountered so far.
- `num in seen` is an O(1) average-case membership test — this single line replaces the entire inner loop from brute force.
- If we make it through the whole array without a match, no duplicates exist, so we return `False`.

✅ Why This Works

A set never stores duplicate values, so checking membership before insertion is exactly equivalent to asking "has this exact value appeared earlier in this traversal?" — which is the definition of a duplicate.

⏱ Complexity Analysis

Approach	Time	Space
Brute Force (nested loops)	O(N²)	O(1)
`count()` per element	O(N²)	O(1)
HashSet (Optimal)	O(N)	O(N)

🎯 Pattern Recognition

🧩 Trigger Words

"Duplicate", "repeated", "frequency", "unique", "seen before" → **HashMap / HashSet**. This is the same family of pattern as Two Sum, Valid Anagram, and Top K Frequent Elements.

⚠ Common Mistakes

- Using `list.count()` inside a loop, thinking it avoids nested loops (it doesn't)
- Sorting first to detect adjacent duplicates — this works but costs O(N log N), which is worse than the O(N) hashset approach
- Forgetting to return `False` when no duplicate is found

🎤 Interview Questions

- **Why is HashSet O(N) and not O(N²)?** Because a set's average-case insert and lookup are both O(1), so N such operations together cost O(N).
- **Could sorting solve this in O(1) extra space?** Yes — sort the array (O(N log N)) and scan for adjacent equal elements (O(N)), giving O(N log N) time and O(1) extra space (a valid space/time trade-off).

🔄 Similar Problems

- 1. Two Sum
- 242. Valid Anagram
- 49. Group Anagrams
- 347. Top K Frequent Elements

"If you've seen it before, it's a duplicate — track 'seen' with a HashSet for O(1) lookups."

Best Time to Buy and Sell Stock

EASY | TRACK MINIMUM SO FAR | LEETCODE 121

📖 Definition

Given the price of a stock on each day, you may buy once and sell once (buy must happen before sell). The goal is to find the maximum possible profit — or 0 if no profit is possible.

💡 Problem Statement

```
Input:  prices = [7,1,5,3,6,4]
Output: 5     (buy at 1, sell at 6)
```

🧠 Intuition

For any selling day, the best possible profit is `current_price - (lowest price seen so far)`. So rather than checking every buy/sell pair, we only need to remember the lowest price encountered up to the current day, and continuously compute the profit if we sold today.

🐢 Brute Force Approach

Idea: Check every possible buy day `i` and sell day `j > i`, and track the maximum profit.

```python
for i in range(len(prices)):
    for j in range(i+1, len(prices)):
        profit = prices[j] - prices[i]
        max_profit = max(max_profit, profit)
```

Time Complexity: O(N²) | **Space Complexity:** O(1)

Why it is inefficient: We recompute profit for every pair, even though most of that information could be reused while scanning forward.

🚀 Optimal Approach — Track Minimum So Far

Main idea: Keep two running values: the minimum price seen so far (`min_price`), and the best profit seen so far (`max_profit`). For every new price, first check the profit if we sold today (`price - min_price`), then update `min_price` if today's price is lower than anything before.

Pattern used: Track Minimum/Maximum So Far — a single forward pass.

Why it is better: Each day's potential profit is computed using only the best buy point so far, so a single O(N) traversal suffices.

🔍 Dry Run

```
prices = [7, 1, 5, 3, 6, 4]
min_price = 7, max_profit = 0

price=7: min_price=7,            profit=0,           max_profit=0
price=1: min_price updates to 1, profit=1-1=0,        max_profit=0
price=5: min_price stays 1,      profit=5-1=4,        max_profit=4
price=3: min_price stays 1,      profit=3-1=2,        max_profit=4
price=6: min_price stays 1,      profit=6-1=5,        max_profit=5
price=4: min_price stays 1,      profit=4-1=3,        max_profit=5

Return max_profit = 5
```

💻 Code

```python
def maxProfit(prices):
    min_price = prices[0]
    max_profit = 0

    for price in prices:
        if price < min_price:
            min_price = price

        profit = price - min_price
        if profit > max_profit:
            max_profit = profit

    return max_profit
```

🔎 Code Explanation

- `min_price` always represents the cheapest "buy day" found up to (and including) the current day.
- `profit = price - min_price` represents the best possible profit if we sold on the current day, using the best buy point found so far.
- `max_profit` simply remembers the best profit found across all days visited.

✅ Why This Works

The maximum profit must come from selling at some day `j` after buying at the lowest price among all days before `j`. By continuously tracking the running minimum as we scan forward, we always have exactly that "best buy price" available at every potential sell day — no need to look backward or recheck.

⏱️ Complexity Analysis

Approach	Time	Space
Brute Force	O(N²)	O(1)
Track Minimum So Far (Optimal)	O(N)	O(1)

A single traversal updates two variables per step — no nested comparisons needed.

🎯 Pattern Recognition

🧩 Trigger Words

"Buy low, sell high", "one transaction", "maximum profit" → **Track Running Minimum/Maximum**. This single-pass tracking idea generalizes to many "best so far" problems.

⚠ Common Mistakes

- Updating `min_price` before computing `profit` for the same day (usually harmless here, but get the order intentional)
- Initializing `max_profit` incorrectly to a negative number instead of 0 (the problem guarantees "0 if no profit possible")
- Confusing this with the "multiple transactions allowed" variant (LeetCode 122), which needs a different approach

🎤 Interview Questions

- **Why does tracking the minimum work instead of checking all pairs?** Because the best profit ending at any day only depends on the cheapest price seen before that day — nothing else matters.
- **What if prices are strictly decreasing?** `max_profit` stays 0 throughout, correctly reflecting that no profitable transaction exists.

🔄 Similar Problems

- 122. Best Time to Buy and Sell Stock II
- 123. Best Time to Buy and Sell Stock III
- 309. Best Time to Buy and Sell Stock with Cooldown
- 53. Maximum Subarray (similar "running state" idea)

"Track the cheapest price seen so far; profit today = today's price minus that minimum."

Binary Search

EASY BINARY SEARCH LEETCODE 704

📖 Definition

Binary Search finds the index of a target value in a **sorted** array by repeatedly cutting the search space in half — comparing the target against the middle element and discarding the half that cannot contain it.

💡 Problem Statement

```
Input:  nums = [-1,0,3,5,9,12], target = 9
Output: 4
```

🧠 Intuition

Because the array is sorted, comparing the target with the middle element instantly tells us which half it could possibly be in: if `target > nums[mid]`, it can only be in the right half; if smaller, only the left half. Every comparison throws away half the remaining elements — that halving is exactly why Binary Search is so fast.

> 💡 **Prerequisite**
>
> The array **must** be sorted, and must support random-access indexing. Binary Search cannot be applied otherwise.

🐢 Brute Force Approach — Linear Search

Idea: Check every element from left to right until the target is found.

```
for i in range(len(nums)):
    if nums[i] == target:
        return i
return -1
```

Time Complexity: O(N) | **Space Complexity:** O(1)

Why it is inefficient: It completely ignores the fact that the array is sorted, checking elements one at a time even when most of them could be safely skipped.

🚀 Optimal Approach — Binary Search

Main idea: Maintain two pointers, `low` and `high`, bounding the current search range. Repeatedly compute `mid`, compare with target, and shrink the range to the half that could still contain it.

Pattern used: Binary Search — eliminate half the search space every iteration.

Why it is better: Each comparison discards half the remaining elements, so the number of comparisons needed grows only logarithmically with N.

🔍 Dry Run

```
nums = [-1, 0, 3, 5, 9, 12], target = 9
low = 0, high = 5

Iteration 1:
  mid = (0+5)//2 = 2, nums[2] = 3
  target(9) > 3  -> discard left half -> low = mid+1 = 3

Iteration 2:
  low = 3, high = 5, mid = (3+5)//2 = 4
  nums[4] = 9  -> TARGET FOUND -> return 4
```

> 🚀 **Why Binary Search Is So Fast**
>
> For 1,000,000 elements: comparison 1 removes 500,000; comparison 2 removes 250,000; comparison 3 removes 125,000... The search space keeps halving, giving **O(log N)** comparisons instead of up to a million.

💻 Code

```python
def search(nums, target):
    low = 0
    high = len(nums) - 1

    while low <= high:
        mid = (low + high) // 2

        if nums[mid] == target:
            return mid
        elif target > nums[mid]:
            low = mid + 1       # search right half
        else:
            high = mid - 1      # search left half

    return -1
```

🔎 Code Explanation

- `low` and `high` always bound the only region where the target could still exist.
- `mid = (low + high) // 2` picks the middle index of the current range.
- The loop condition `low <= high` ensures the last remaining single element still gets checked before giving up.
- Whichever side cannot contain the target is excluded by moving `low` or `high` past `mid`.

✅ Why This Works

Because the array is sorted, every element to the left of `mid` is ≤ `nums[mid]` and every element to the right is ≥ `nums[mid]`. So if the target is greater than `nums[mid]`, it is mathematically impossible for it to exist anywhere in the left half — it is safe to discard entirely.

⏱ Complexity Analysis

Case	Time
Best Case (found at first mid)	O(1)
Average Case	O(log N)
Worst Case	O(log N)

Space Complexity: O(1) — only a few pointer variables are used, no extra data structures.

🎯 Pattern Recognition

🧩 Trigger Words

"Sorted array", "search element", "minimize/maximize in a sorted answer space", "need O(log N)" → **Binary Search**.

⚠ Common Mistakes

- `mid = low + high // 2` — operator precedence makes this `low + (high // 2)`, which is wrong. Always write `mid = (low + high) // 2`.
- Using `while low < high` instead of `while low <= high` — this can skip checking the last remaining element.
- Applying Binary Search on an unsorted array — it silently gives wrong answers instead of erroring out.

🎤 Interview Questions

- **Why is Binary Search O(log N)?** Because every comparison eliminates half of the remaining search space, so the number of halvings needed to shrink N elements down to 1 is $\log_2 N$.
- **Can Binary Search work on unsorted arrays?** No — the sorted property is mandatory because it's what guarantees one side can be safely discarded.
- **Why use `low <= high` instead of `low < high`?** Because the last remaining single element (when `low == high`) still needs to be checked.

🔄 Similar Problems

- 35. Search Insert Position

- 34. Find First and Last Position of Element in Sorted Array
- 33. Search in Rotated Sorted Array
- 69. Sqrt(x)
- 875. Koko Eating Bananas

"Binary Search works only on sorted arrays. Compare with the middle, then eliminate half — repeat until found."

Running Sum of 1D Array

EASY PREFIX SUM LEETCODE 1480

📖 Definition

The running sum at index `i` is the sum of all elements from index 0 up to and including `i`. This is our first introduction to the **Prefix Sum** pattern.

💡 Problem Statement

```
Input:  nums = [1,2,3,4]
Output: [1,3,6,10]

Index 0 -> 1
Index 1 -> 1+2 = 3
Index 2 -> 1+2+3 = 6
Index 3 -> 1+2+3+4 = 10
```

🧠 Intuition

Every running sum depends only on the *previous* running sum plus the current element: `runningSum[i] = runningSum[i-1] + nums[i]`. There is no need to re-add everything from scratch each time — we can reuse the work already done.

🐢 Brute Force Approach

Idea: For every index, start from index 0 and re-sum everything up to that index.

```
for i in range(len(nums)):
    total = 0
    for j in range(i + 1):
        total += nums[j]
    result.append(total)
```

Time Complexity: O(N²) | **Space Complexity:** O(N)

Why it is inefficient: The sum up to index 2 is recomputed from scratch when calculating index 3, even though it was already known.

🚀 Optimal Approach — In-Place Prefix Sum

Main idea: Starting from index 1, add the previous element's (already updated) value into the current element. Since index 0 has no element before it, its running sum is just itself.

Pattern used: Prefix Sum (build once, reuse the running total).

Why it is better: A single forward pass computes every running sum using only the immediately preceding result.

🔍 Dry Run

```
nums = [1, 2, 3, 4]

i=1: nums[1] += nums[0]  -> 2+1=3   -> [1, 3, 3, 4]
i=2: nums[2] += nums[1]  -> 3+3=6   -> [1, 3, 6, 4]
i=3: nums[3] += nums[2]  -> 4+6=10  -> [1, 3, 6, 10]

Return [1, 3, 6, 10]
```

💻 Code

```python
def runningSum(nums):
    i = 1
    while i < len(nums):
        nums[i] += nums[i - 1]
        i += 1
    return nums
```

🔎 Code Explanation

- We start the loop at index 1 because index 0 has no "previous" sum to add — it is already its own running sum.
- `nums[i] += nums[i-1]` works because by the time we reach index `i`, `nums[i-1]` has already been transformed into the correct running sum (not the original value).
- The array is modified in-place, eliminating the need for a separate result array.

✅ Why This Works

Mathematically, `sum(0..i) = sum(0..i-1) + nums[i]`. Since the algorithm processes indices left to right and overwrites each cell with its true running sum before moving on, the recurrence is always satisfied using correct, already-computed values.

⏱ Complexity Analysis

Approach	Time	Space
Brute Force	O(N²)	O(N)
In-Place Prefix Sum (Optimal)	O(N)	O(1)

🎯 Pattern Recognition

🧩 Trigger Words

"Running sum", "cumulative sum", "current answer depends on all previous elements" → **Prefix Sum**.

⚠ Common Mistakes

- Starting the loop from index 0 — `nums[0] += nums[-1]` wraps around to the last element in Python, producing a wrong answer.
- Using `while i <= len(nums)` instead of `<` — causes an out-of-bounds index.
- Printing the result instead of returning it.

🎤 Interview Questions

- **Why start from index 1?** Because the first element has no previous element to add.
- **Why modify the array in-place?** To achieve O(1) extra space instead of O(N) for a separate result array.

🔄 Similar Problems

- 724. Find Pivot Index
- 303. Range Sum Query – Immutable
- 238. Product of Array Except Self

"Running Sum = Current Element + Previous Running Sum, built in a single forward pass."

Range Sum Query – Immutable

EASY PREFIX SUM LEETCODE 303

📖 Definition

Given an array and many range-sum queries, we must answer `sumRange(left, right)` — the sum of elements from index `left` to `right` inclusive — as quickly as possible for each query.

💡 Problem Statement

```
nums = [-2,0,3,-5,2,-1]
sumRange(2, 5) = 3 + (-5) + 2 + (-1) = -1
```

🧠 Intuition

If there will be thousands of queries, recomputing each sum from scratch is wasteful. Instead, build a **prefix sum array** once — where `prefix[i]` stores the sum of everything from index 0 to `i` — then answer every future query using subtraction in O(1).

🐢 Brute Force Approach

Idea: For each query, loop from `left` to `right` and sum directly.

```
def sumRange(self, left, right):
    total = 0
    for i in range(left, right + 1):
        total += self.nums[i]
    return total
```

Time Complexity: O(N) per query | **Space Complexity:** O(1)

Why it is inefficient: With many overlapping queries, the same sub-ranges get summed again and again — pure repeated work.

🚀 Optimal Approach — Build Once, Query in O(1)

Main idea: Build the prefix sum array once in the constructor. Then, `sumRange(left, right) = prefix[right] - prefix[left-1]` (with a special case when `left == 0`).

Pattern used: Prefix Sum — precompute once, answer many queries instantly.

Why it is better: Building the prefix array costs O(N) total, but each query afterward costs only O(1).

🔍 Why `prefix[right] - prefix[left-1]` ?

`prefix[right]` contains the sum of everything from index 0 to `right`. To get only the sum from `left` to `right`, we must remove everything *before* `left` — and that "everything before" is exactly `prefix[left-1]`.

```
nums    = [2, 4, 6, 8, 10]
prefix  = [2, 6, 12, 20, 30]

sumRange(2, 4) wants: 6+8+10 = 24

prefix[4] = 30   (contains 2+4+6+8+10)
prefix[1] = 6    (contains 2+4, i.e. everything BEFORE index 2)

Answer = prefix[4] - prefix[1] = 30 - 6 = 24  ✔
```

⚠ Why not `prefix[left]` instead of `prefix[left-1]` ?

`prefix[left]` already *includes* the element at `left`. Subtracting it would accidentally remove the left boundary element from the answer too. We must subtract `prefix[left-1]` to exclude only the elements strictly before `left`.

💻 Code

```python
class NumArray:
    def __init__(self, nums):
        self.prefix = nums[:]
        for i in range(1, len(self.prefix)):
            self.prefix[i] += self.prefix[i - 1]

    def sumRange(self, left, right):
        if left == 0:
            return self.prefix[right]
        return self.prefix[right] - self.prefix[left - 1]
```

🔎 Code Explanation

- `__init__` runs exactly once when the object is created, building the prefix array up front.
- `sumRange` never rebuilds anything — it only performs O(1) arithmetic using the existing prefix array.
- The special case `left == 0` exists because there is no "index -1" to subtract — there's simply nothing before the start.

✅ Why This Works

By definition, `Total = Left + Right` where `Total = prefix[right]` and `Left = prefix[left-1]`. Rearranging gives `Right = Total - Left`, exactly the formula used.

⏱️ Complexity Analysis

Operation	Brute Force	Prefix Sum (Optimal)
Build	—	O(N)
Each Query	O(N)	O(1)
Space	O(1)	O(N)

🎯 Pattern Recognition

🧩 Trigger Words

"Many range sum queries", "immutable array", "answer queries quickly" → **Prefix Sum**. Build once, answer many times.

⚠ Common Mistakes

- Using `prefix[left]` instead of `prefix[left-1]` (double counts or excludes the boundary)
- Forgetting the `left == 0` special case
- Rebuilding the prefix array inside `sumRange()` — defeats the entire purpose

🎤 Interview Questions

- **Why use Prefix Sum here?** To answer many range queries quickly without repeating work each time.
- **Why subtract `prefix[left-1]` and not `prefix[left]`?** Because we only want to exclude elements strictly before `left`, not `left` itself.
- **Why is this called "Immutable"?** Because the array never changes after construction — if updates were allowed, a Segment Tree or Fenwick Tree would be needed instead.

🔄 Similar Problems

- 1480. Running Sum of 1D Array
- 724. Find Pivot Index
- 304. Range Sum Query 2D – Immutable
- 307. Range Sum Query – Mutable (needs Segment Tree)

"Build the Prefix Sum once. Range Sum = prefix[right] - prefix[left-1]."

Find Pivot Index

EASY PREFIX SUM LEETCODE 724

📖 Definition

The Pivot Index is the index where the sum of all elements strictly to its left equals the sum of all elements strictly to its right. The pivot element itself is excluded from both sums.

💡 Problem Statement

```
Input:  nums = [1,7,3,6,5,6]
Output: 3

Left Side  (indices 0-2): 1+7+3 = 11
Right Side (indices 4-5): 5+6   = 11
Equal -> Pivot Index = 3
```

🧠 Intuition

If we already know the **total sum** of the array and the **running left sum** up to (but not including) the current index, then: `right_sum = total_sum - left_sum - nums[i]`. This avoids recomputing left and right sums from scratch at every index.

🐢 Brute Force Approach

Idea: For every index, separately compute the left sum and right sum using slicing, and compare.

```python
for i in range(len(nums)):
    left_sum = sum(nums[:i])
    right_sum = sum(nums[i+1:])
    if left_sum == right_sum:
        return i
return -1
```

Time Complexity: O(N²) | **Space Complexity:** O(1)

Why it is inefficient: `sum()` re-scans a sub-array on every iteration — for index 4, Python recalculates `1+7+3+6` all over again, even though it was already known one step earlier.

🚀 Optimal Approach — Prefix Sum via Total - Left - Current

Main idea: Compute the array's `total_sum` once. Then traverse left to right, maintaining a running `left_sum`. At each index, derive `right_sum` using the formula above, compare, and only *then* fold the current element into

`left_sum`.

Pattern used: Prefix Sum (derived, without building a full prefix array).

Why it is better: Both `total_sum` and `left_sum` are maintained incrementally, so each index is processed in O(1), giving O(N) overall.

🔍 Dry Run

```
nums = [1,7,3,6,5,6],  total_sum = 28,  left_sum = 0

i=0: right = 28-0-1 = 27   -> left(0) != right(27)  -> left_sum += 1 -> 1
i=1: right = 28-1-7 = 20   -> left(1) != right(20)  -> left_sum += 7 -> 8
i=2: right = 28-8-3 = 17   -> left(8) != right(17)  -> left_sum += 3 -> 11
i=3: right = 28-11-6 = 11  -> left(11) == right(11) -> return 3
```

💻 Code

```python
def pivotIndex(nums):
    total_sum = sum(nums)
    left_sum = 0

    for i in range(len(nums)):
        right_sum = total_sum - left_sum - nums[i]

        if left_sum == right_sum:
            return i

        left_sum += nums[i]

    return -1
```

🔎 Code Explanation

- `total_sum` is computed only once, up front.
- `right_sum` is derived using `total - left - current`, since `Total = Left + Current + Right` by definition.
- We compare *before* updating `left_sum`, because the current element must not be counted as part of the left side.
- `left_sum += nums[i]` happens only after the check, correctly preparing the running sum for the next index.

✅ Why This Works

By definition, every element of the array belongs to exactly one of three groups for a given index `i`: the left group, the current element, or the right group. So `Total = Left + Current + Right`, which directly rearranges to `Right = Total - Left - Current`.

⏱️ Complexity Analysis

Approach	Time	Space
Brute Force (slicing + sum())	O(N²)	O(1)
Derived Prefix Sum (Optimal)	O(N)	O(1)

🎯 Pattern Recognition

🧩 Trigger Words

"Left sum equals right sum", "repeated sum calculations" → **Prefix Sum**, often without needing to materialize a full prefix array.

⚠ Common Mistakes

- Updating `left_sum` before checking equality — this incorrectly includes the current element on the left side.
- Calling `sum()` inside the loop — silently reintroduces O(N²) time.
- Forgetting to `return -1` when no pivot index exists.

🎤 Interview Questions

- **Why calculate `total_sum` only once?** To avoid repeating the same O(N) summation at every index.
- **Why update `left_sum` after checking, not before?** Because the current element does not belong to the left side at the moment of comparison.
- **Can this be solved without a full Prefix Array?** Yes — by maintaining just the running total and left sum, as shown here, achieving O(1) extra space.

🔄 Similar Problems

- 1480. Running Sum of 1D Array
- 303. Range Sum Query – Immutable
- 1991. Find the Middle Index in Array

"Right = Total - Left - Current. Compare, then update Left — in that exact order."

Maximum Subarray

EASY KADANE'S ALGORITHM LEETCODE 53

📖 Definition

Given an integer array, find the contiguous subarray (consecutive elements only) with the largest possible sum, and return that sum.

💡 Problem Statement

```
Input:  nums = [-2,1,-3,4,-1,2,1,-5,4]
Output: 6      (subarray [4,-1,2,1])
```

Key Concept — Contiguous: `[4,-1,2]` is allowed (consecutive); `[4,2]` is not (skips an element).

🧠 Intuition

Think of carrying a backpack while walking through the array. Positive numbers are gold (keep carrying); a string of negative numbers is a heavy rock weighing you down. If your current load (running sum) becomes more of a burden than starting fresh, **drop it and restart from the current element**. That's the entire idea behind Kadane's Algorithm.

🐢 Brute Force Approach

Idea: Generate every possible subarray (every start index, every end index) and track the maximum sum found.

```
max_sum = nums[0]
for i in range(len(nums)):
    current_sum = 0
    for j in range(i, len(nums)):
        current_sum += nums[j]
        max_sum = max(max_sum, current_sum)
return max_sum
```

Time Complexity: O(N²) | **Space Complexity:** O(1)

Why it is inefficient: Every starting index triggers a fresh inner scan — for large arrays this becomes far too slow (TLE).

🚀 Optimal Approach — Kadane's Algorithm

Main idea: At every element, ask only one question: *"Should I continue my current subarray, or restart from here?"* Take whichever choice gives a larger sum: `current_sum = max(num, current_sum + num)`. Track the best `current_sum` ever seen as the overall answer.

Pattern used: Kadane's Algorithm — single-pass running state with a "continue vs restart" decision.

Why it is better: Each element is visited exactly once, replacing the entire brute-force double loop with one decision per step.

🔍 Dry Run

```
nums = [5, -10, 6]
current_sum = 5, max_sum = 5

num=-10: continue = 5+(-10) = -5;  restart = -10;  choose -5
         current_sum = -5,  max_sum stays 5

num=6:   continue = -5+6 = 1;     restart = 6;     choose 6
         current_sum = 6,  max_sum updates to 6

Return max_sum = 6
```

💻 Code

```python
def maxSubArray(nums):
    current_sum = nums[0]
    max_sum = nums[0]

    for num in nums[1:]:
        current_sum = max(num, current_sum + num)
        max_sum = max(max_sum, current_sum)

    return max_sum
```

🔎 Code Explanation

- `current_sum` represents the best subarray sum that ends exactly at the current element.
- `max(num, current_sum + num)` is the "continue vs restart" decision — pick whichever gives a bigger running total.
- `max_sum` tracks the best `current_sum` seen across the entire traversal, since the optimal subarray could end at any position.

✅ Why This Works

A negative running sum can never help a future subarray — adding more elements on top of a negative total only makes things worse. So whenever continuing would produce a smaller result than restarting fresh, restarting is strictly better. This greedy local decision, repeated at every index, provably yields the global maximum.

⏱️ Complexity Analysis

Approach	Time	Space
Brute Force	$O(N^2)$	O(1)
Kadane's Algorithm (Optimal)	O(N)	O(1)

🎯 Pattern Recognition

🧩 Trigger Words

"Maximum sum", "contiguous subarray", "largest sum" → **Kadane's Algorithm**.

⚠ Common Mistakes

- Checking only `nums[i] + nums[j]` — ignores all elements in between.
- Initializing `max_sum = 0` — fails for all-negative arrays (e.g. `[-5,-2,-8]` should return -2, not 0).
- Always writing `current_sum += num` with no restart option — this removes Kadane's core strength.

🎤 Interview Questions

- **Why does Kadane restart instead of always continuing?** Because a negative running sum can never help future elements, so dropping it and starting fresh is always at least as good.
- **What if the entire array is negative?** The algorithm still works correctly, returning the single largest (least negative) element.

🔄 Similar Problems

- 152. Maximum Product Subarray
- 121. Best Time to Buy and Sell Stock
- 918. Maximum Sum Circular Subarray
- 1186. Maximum Subarray Sum with One Deletion

"At every step: Continue = current_sum + num, Restart = num. Take the max, then update the global answer."

Maximum Product Subarray

MEDIUM | KADANE'S VARIATION (TRACK MIN & MAX) | LEETCODE 152

📖 Definition

Find the contiguous subarray within an array that has the largest product, and return that product.

💡 Problem Statement

```
Input:  nums = [2,3,-2,4]
Output: 6      (subarray [2,3])
```

🧠 Intuition — Why Plain Kadane Fails Here

For **sums**, a negative running total is always bad, so we restart. But for **products**, a negative running product is *not* always bad — multiplying it by another negative number can flip it into the largest positive product.

```
Sum case:      current_sum = -20, next = 5  -> -20+5 = -15  (worse than restart=5)
Product case:  current_prod = -20, next = -5 -> -20*(-5) = 100 (huge win!)
```

So we can never simply discard a negative product — it might be exactly what we need later.

🚀 Core Observation

Negative × Negative = Positive. This means today's **smallest (most negative)** product could become tomorrow's **largest** product. Therefore, at every step, we must track **both**:

- **max_prod** — the maximum product ending at the current element
- **min_prod** — the minimum (most negative) product ending at the current element

🐢 Brute Force Approach

Idea: Generate every possible contiguous subarray, compute its product, and track the maximum.

Time Complexity: O(N²) | **Space Complexity:** O(1)

Why it is inefficient: Same issue as Maximum Subarray's brute force — repeated work recomputing products for overlapping ranges.

🚀 Optimal Approach — Track Both Max and Min

Main idea: At every element, there are **three** possibilities for the best product ending here: start fresh with `num`, extend the previous maximum (`max_prod * num`), or extend the previous minimum (`min_prod * num`). Take the

largest of the three as the new `max_prod`, and the smallest of the three as the new `min_prod`.

Pattern used: Kadane's Algorithm Variation — dual-state tracking (max AND min).

Why it is better: A single O(N) pass correctly accounts for sign flips, which a single-variable Kadane cannot.

> 🚀 **Why Use Temporary Variables?**
>
> If `max_prod` is updated *before* computing the new `min_prod`, the min calculation will incorrectly use the already-updated (new) value instead of the old one. Always compute `temp_max` and `temp_min` first using the OLD values, then assign both at once.

🔍 Dry Run

```
nums = [2, 3, -2, 4]
max_prod = 2, min_prod = 2, answer = 2

num=3:  choices: 3, 2*3=6, 2*3=6      -> temp_max=6,  temp_min=3
        max_prod=6, min_prod=3, answer=6

num=-2: choices: -2, 6*-2=-12, 3*-2=-6 -> temp_max=-2, temp_min=-12
        max_prod=-2, min_prod=-12, answer=6

num=4:  choices: 4, -2*4=-8, -12*4=-48 -> temp_max=4,  temp_min=-48
        max_prod=4, min_prod=-48, answer=6

Return answer = 6
```

💻 Code

```python
def maxProduct(nums):
    max_prod = nums[0]
    min_prod = nums[0]
    answer = nums[0]

    for num in nums[1:]:
        temp_max = max(num, max_prod * num, min_prod * num)
        temp_min = min(num, max_prod * num, min_prod * num)

        max_prod = temp_max
        min_prod = temp_min

        answer = max(answer, max_prod)

    return answer
```

🔎 Code Explanation

- `temp_max` evaluates all three possibilities (restart, extend max, extend min) and keeps the largest.

- `temp_min` does the same but keeps the smallest — this is essential for "remembering" a deeply negative product that might flip positive later.
- Both temporaries are computed from the *old* `max_prod` / `min_prod` before either is overwritten, avoiding the order-of-update bug.
- `answer` tracks the best `max_prod` seen across the whole traversal.

✅ Why This Works

Because multiplying by a negative number swaps the relative order of the maximum and minimum values, only tracking the maximum (as plain Kadane does) would lose information whenever a negative number appears. Tracking both extremes guarantees that whichever value can become the new maximum after a sign flip is always available.

⏱ Complexity Analysis

Approach	Time	Space
Brute Force	O(N²)	O(1)
Track Max & Min (Optimal)	O(N)	O(1)

🎯 Pattern Recognition

> **🧩 Trigger Words**
> "Maximum Product", "contiguous", "subarray" → Think: track **BOTH** maximum and minimum product, not just maximum.

Maximum Subarray vs Maximum Product Subarray

Maximum Subarray (LC 53)	Maximum Product Subarray (LC 152)
Tracks only `current_sum`	Tracks `max_prod` AND `min_prod`
Two choices: Continue or Restart	Three choices: Start New, Extend Max, Extend Min

> **⚠ Common Mistakes**
> - Applying plain Kadane's sum logic directly to products — it only works for sums.
> - Tracking only the maximum product and ignoring the minimum.
> - Updating `max_prod` before computing `min_prod` — always use temporary variables.
> - Forgetting that a single zero resets both `max_prod` and `min_prod` to zero, effectively "restarting" the subarray.

🎤 Interview Questions

- **Why do we need to track the minimum product too?** Because a very negative product can become the largest positive product after multiplying by another negative number.
- **Why use temp_max/temp_min instead of updating directly?** To ensure both new values are computed from the same old state, avoiding incorrect use of an already-updated variable.

🔄 Similar Problems

- 53. Maximum Subarray
- 238. Product of Array Except Self
- 713. Subarray Product Less Than K

"Negative × Negative = Positive — always track both the max AND min product ending at each index."

Merge Sorted Array

EASY | FILL FROM BACK / TWO POINTERS | LEETCODE 88

📖 Definition

Given two sorted arrays where the first array (`nums1`) has extra trailing space exactly large enough to hold all elements of the second array (`nums2`), merge them in-place into one sorted array.

💡 Problem Statement

```
Input:  nums1 = [1,2,3,0,0,0], m = 3
        nums2 = [2,5,6],       n = 3
Output: [1,2,2,3,5,6]
```

Important: the zeros in `nums1` are not real data — they're just empty placeholder slots reserved for merging.

🧠 Intuition

Since both arrays are already sorted, we should never re-sort anything — we should merge them directly, the same way merge sort's "merge" step works. The twist here is the extra space: it sits at the **end** of `nums1` , which is a strong hint to fill the result from the back rather than the front.

🐢 Brute Force Approach

Idea: Extract the valid elements of `nums1` , combine with `nums2` , sort everything, then copy back.

```
temp = nums1[:m]
combined = temp + nums2
combined.sort()
for i in range(len(combined)):
    nums1[i] = combined[i]
```

Time Complexity: O((m+n) log(m+n)) | **Space Complexity:** O(m+n)

Why it is inefficient: Sorting from scratch completely ignores the fact that both arrays are already individually sorted — wasted work, plus extra memory.

🚀 Optimal Approach — Fill From the Back

Main idea: Use three pointers: `i` at the last valid element of `nums1` , `j` at the last element of `nums2` , and `k` at the last available slot in `nums1` . Compare `nums1[i]` and `nums2[j]` , place the larger one at `nums1[k]` , and move the corresponding pointer inward. Filling from the back guarantees we never overwrite a value we still need to read.

Pattern used: Two Pointers / Merge — Fill From the Back.

Why it is better: Zero extra memory, and every element is processed exactly once — O(m+n) time, O(1) space.

🔍 Dry Run

```
nums1 = [1,2,3,0,0,0],  nums2 = [2,5,6]
i = 2 (value 3), j = 2 (value 6), k = 5

Compare nums1[2]=3 vs nums2[2]=6 -> 6 is larger
  place 6 at nums1[5]  -> [1,2,3,0,0,6],  j=1, k=4

Compare nums1[2]=3 vs nums2[1]=5 -> 5 is larger
  place 5 at nums1[4]  -> [1,2,3,0,5,6],  j=0, k=3

Compare nums1[2]=3 vs nums2[0]=2 -> 3 is larger
  place 3 at nums1[3]  -> [1,2,3,3,5,6],  i=1, k=2

Compare nums1[1]=2 vs nums2[0]=2 -> equal, place nums2's 2
  place 2 at nums1[2]  -> [1,2,2,3,5,6],  j=-1, k=1

j < 0, stop main loop -> DONE
Final: [1,2,2,3,5,6]
```

💻 Code

```python
def merge(nums1, m, nums2, n):
    i = m - 1
    j = n - 1
    k = m + n - 1

    while i >= 0 and j >= 0:
        if nums1[i] > nums2[j]:
            nums1[k] = nums1[i]
            i -= 1
        else:
            nums1[k] = nums2[j]
            j -= 1
        k -= 1

    # Copy any remaining elements of nums2
    while j >= 0:
        nums1[k] = nums2[j]
        j -= 1
        k -= 1
```

🔎 Code Explanation

- `i`, `j`, and `k` all start at the rightmost relevant position of their respective arrays.
- The main loop places the larger of the two current candidates into the last open slot, working backward.
- The second `while j >= 0` loop handles the case where `nums2` still has smaller leftover elements after `nums1`'s valid elements run out — these must still be copied in.
- There is deliberately **no** equivalent `while i >= 0` loop, because any remaining elements of `nums1` are already exactly where they need to be.

✅ Why This Works

Filling from the back means we always write into a slot at index `k`, which is always ≥ both `i` and `j`. Since we only ever read from `i` or `j` (both strictly less than or equal to `k`), we never overwrite a value before we've had the chance to read it.

⏱️ Complexity Analysis

Approach	Time	Space
Sort Combined Array	O((m+n) log(m+n))	O(m+n)
Fill From Back (Optimal)	O(m+n)	O(1)

🎯 Pattern Recognition

🧩 Trigger Words

"Two sorted arrays", "merge", "in-place", "extra space at the end" → **Fill From the Back**.

⚠ Common Mistakes

- Filling from the front — this overwrites valid elements of `nums1` before they've been compared.
- Re-sorting when both arrays are already individually sorted — unnecessary extra time complexity.
- Forgetting the second `while j >= 0` loop, leaving some of `nums2`'s elements uncopied.
- Treating the trailing zeros in `nums1` as real data instead of empty placeholders.

🎤 Interview Questions

- **Why fill from the end instead of the front?** Because the extra space lives at the end, so writing backward guarantees nothing gets overwritten before it's read.
- **Why is there no "remaining nums1" loop?** Any leftover elements of `nums1` are already in their correct final positions once the main loop finishes.
- **What's the time complexity and why?** O(m+n), because every element from both arrays is visited and placed exactly once.

🔄 Similar Problems

- 21. Merge Two Sorted Lists
- 977. Squares of a Sorted Array
- 350. Intersection of Two Arrays II
- 23. Merge k Sorted Lists

"Start from the last valid elements of both arrays, compare, place the larger at the last empty slot, and work backward."

📊 Reference & Revision

The rest of this handbook is a fast-access reference layer — built for revision the night before an interview, not for first-time learning. Flip here whenever you need a quick reminder instead of re-reading a full topic.

📊 Complete Complexity Cheat Sheet

Complexity	Definition	Real Example	Typical Algorithms	Memory Trick
O(1)	Time doesn't change with input size	`arr[5]`	Array indexing, hashmap lookup, push/pop on a stack	"Instant — no matter how big N is"
O(log N)	Time grows by 1 step every time N *doubles*	Halving a search space	Binary Search, balanced BST operations	"Cut in half, again and again"
O(N)	Time grows directly proportional to N	One full traversal	Linear Search, single-pass HashMap solutions	"One pass, one touch per element"
O(N log N)	N times a logarithmic factor	Sorting	Merge Sort, Quick Sort (avg), Heap Sort	"Sort-speed — the best a comparison sort can do"
O(N²)	Time grows with the square of N	Nested loops, all-pairs comparison	Bubble Sort, brute-force pair search, triangle loops	"Every element checks every other element"
O(2^N)	Time doubles with every additional element	Generating all subsets	Brute-force subset/combination generation, naive recursion (e.g. Fibonacci)	"Every element doubles the work — explosive"
O(N!)	Time grows factorially	Generating all permutations	Brute-force Traveling Salesman, permutation generation	"Every arrangement of everything — worst possible blow-up"

Common Interview Questions on Complexity

- **Why does Binary Search beat Linear Search so dramatically at scale?** Because O(log N) barely grows even as N becomes huge, while O(N) grows linearly with it.
- **Is O(N) always faster than O(N log N) in practice?** Asymptotically yes for large N, but for small N the constants involved can make either faster — Big-O describes trend, not a guaranteed real-world race.
- **When is O(N²) acceptable?** When N is small/bounded (e.g. N ≤ 1000) and a simpler O(N²) approach is easier to write correctly under time pressure.

"O(1) < O(log N) < O(N) < O(N log N) < O(N²) < O(2^N) < O(N!) — memorize the order, not just the symbols."

🧩 Pattern Recognition Guide

Use this table the moment you read a new problem. Match the keywords in the question to the pattern, before writing a single line of code.

If the question contains...	Think...	Seen In
Pair Sum / "two numbers that add up to"	HashMap (complement technique)	Two Sum
Duplicate / Repeated / Seen Before	HashSet	Contains Duplicate
Maximum Sum / Contiguous / Subarray	Kadane's Algorithm	Maximum Subarray
Maximum Product / Contiguous	Kadane's Variation — track Max AND Min	Maximum Product Subarray
Sorted Array / Search Element / need O(log N)	Binary Search	Binary Search
Continuous Range Sum / Many Queries	Prefix Sum	Running Sum, Range Sum Query, Pivot Index
Rotate / Shift by k	Reverse Technique or Modulo Index Mapping	Rotate Array
Merge Sorted Arrays / In-place / Extra space at end	Fill From the Back	Merge Sorted Array
Move/Group elements while keeping relative order	Two Pointers (fast scanner / slow writer)	Move Zeroes
Reverse / Swap mirror positions	Two Pointers (converging from both ends)	Reverse Array
Buy low, sell high / One Transaction	Track Running Minimum/Maximum	Best Time to Buy and Sell Stock

> **🧩 How to Use This Table**
>
> Read the problem once. Underline the nouns and verbs (Sum, Sorted, Duplicate, Rotate, Merge...). Match against this table before thinking about code. Pattern recognition is 80% of solving unseen problems quickly.

"Keywords lead to patterns. Patterns lead to code. Never start coding before naming the pattern."

🐍 Python DSA Shortcuts

Shortcut	Purpose	Syntax	Interview Usage
`enumerate()`	Get index + value together while looping	`for i, v in enumerate(arr):`	HashMap problems needing both value and index (Two Sum)
`zip()`	Loop over two arrays in parallel	`for a, b in zip(arr1, arr2):`	Comparing or merging two equal-length arrays
`sorted()`	Returns a new sorted list (original unchanged)	`sorted(arr)`	Anagram checks, sorting before two-pointer techniques
`.sort()`	Sorts the list in-place (no copy)	`arr.sort()`	O(1) extra space sorting requirement
`reversed()`	Iterate a sequence backward	`for x in reversed(arr):`	Right-to-left scans, palindrome checks
`set()`	O(1) average membership checks	`seen = set()`	Contains Duplicate, "seen before" tracking
`dict.get()`	Safe dictionary lookup with a default	`d.get(key, 0)`	Frequency counting without KeyError
`Counter`	Build a frequency map in one line	`from collections import Counter; Counter(arr)`	Anagram, frequency, majority-element problems
`defaultdict`	Dictionary with automatic default values	`from collections import defaultdict; d = defaultdict(list)`	Grouping problems (Group Anagrams)
`heapq`	Min-heap operations	`import heapq; heapq.heappush(h, x)`	Top-K, Kth largest/smallest, scheduling problems
`deque`	O(1) append/pop from both ends	`from collections import deque; dq = deque()`	Sliding window, BFS
`bisect`	Binary search insert position in a sorted list	`import bisect; bisect.bisect_left(arr, x)`	Search Insert Position, maintaining sorted order on insert
`itertools`	Combinations, permutations, products	`from itertools import combinations`	Subset/combination generation problems

`functools.lru_cache`	Automatic memoization for recursive functions	`from functools import lru_cache; @lru_cache(None)`	Top-down Dynamic Programming
`math.inf`	Represents infinity for comparisons	`import math; best = math.inf`	Initializing "running minimum" trackers safely

"Python's built-in tools often replace 5-10 lines of manual logic with one readable line — know them cold."

🎯 Interview Cheat Sheet (One-Pager)

Big-O Ranking

```
O(1) < O(log N) < O(N) < O(N log N) < O(N²) < O(2^N) < O(N!)
```

Binary Search Conditions

- Array must be **sorted**
- Loop condition: `while low <= high`
- `mid = (low + high) // 2` — always parenthesize the sum first

Prefix Sum Formulas

- `runningSum[i] = runningSum[i-1] + nums[i]`
- `sumRange(left, right) = prefix[right] - prefix[left-1]` (use `prefix[right]` alone if `left == 0`)
- `right_sum = total_sum - left_sum - nums[i]`

Kadane's Algorithm

- Sum version: `current_sum = max(num, current_sum + num)`
- Product version: track BOTH `max_prod` and `min_prod` using temp variables

HashMap / HashSet Tricks

- Two Sum: `complement = target - num`, check before inserting
- Contains Duplicate: `if num in seen: return True` else add to set

Two Pointer Rules

- Reverse: `left, right` start at both ends, move inward, stop at `left < right`
- Move Zeroes: `right` pointer marks the next free slot for non-zero values

Merge Pointers

- Always merge sorted arrays from the **back** when extra space sits at the end
- 3 pointers: `i` (end of array 1), `j` (end of array 2), `k` (end of merged slot)

Rotate Trick

- `k = k % n` first, always
- Reverse Method: reverse all → reverse first k → reverse remaining n-k

Edge Cases to Always Check

- Empty array / single-element array
- All-negative arrays (Maximum Subarray, Best Time to Buy/Sell Stock)
- k larger than array length (Rotate Array)
- Target not present (Binary Search, Two Sum)
- Duplicate values (Two Sum, Contains Duplicate)

In-Place Modification Rules

- Always check: are you overwriting a value before you've read it? If yes — change direction (front-to-back vs back-to-front).
- When extra space exists at the end of an array, that's a strong signal to fill from the back.

"This one page should be your last 2-minute glance before walking into the interview room."

⚡ 5-Minute Final Revision Sheet

Read top to bottom. No explanations — only triggers, formulas, and complexities.

Topic	Trigger	Core Formula / Idea	Optimal Complexity
Big-O	Analyzing any code	Drop constants, keep highest term	—
Reverse Array	"Reverse", in-place	Swap left/right, move inward	O(N) / O(1)
Rotate Array	"Rotate by k"	Reverse all → reverse first k → reverse rest	O(N) / O(1)
Move Zeroes	"Move to end, preserve order"	`right` pointer = next free slot	O(N) / O(1)
Two Sum	"Pair sums to target"	`complement = target - num`	O(N) / O(N)
Contains Duplicate	"Any value appears twice"	`if num in seen: return True`	O(N) / O(N)
Best Time to Buy/Sell Stock	"Max profit, one transaction"	`profit = price - min_price`	O(N) / O(1)
Binary Search	"Sorted array, search target"	Compare with mid, eliminate half	O(log N) / O(1)
Running Sum	"Cumulative sum"	`nums[i] += nums[i-1]`	O(N) / O(1)
Range Sum Query	"Many range queries"	`prefix[r] - prefix[l-1]`	Build O(N), Query O(1)
Pivot Index	"Left sum = Right sum"	`right = total - left - nums[i]`	O(N) / O(1)
Maximum Subarray	"Max contiguous sum"	`current = max(num, current+num)`	O(N) / O(1)
Maximum Product Subarray	"Max contiguous product"	Track BOTH max_prod & min_prod	O(N) / O(1)
Merge Sorted Array	"Two sorted, extra space at end"	Fill from the back, 3 pointers	O(m+n) / O(1)

Final Reminders

- Always state Brute Force complexity before jumping to the optimal solution — interviewers want to see the thought process.
- Always mention space complexity, not just time.
- Always check edge cases out loud: empty array, single element, all-negative, duplicates, target absent.
- When stuck, ask: "Have I seen this value before?" (HashMap), "Is this sorted?" (Binary Search), "Is this asking about a running/cumulative total?" (Prefix Sum).

"Pattern first. Brute force second. Optimize third. Edge cases always."